Architecture Checkpoint

Architectural Evolution of nopCommerce

Scenario C — Omnichannel Commerce Core

Abel Teixeira (113655) · Francisco Costa (131224) · Guilherme Pinho (131690) · Tiago Albuquerque (112901)

Software Architectures · UA-DETI · May 2026

Index

01 · SCENARIO

Why Omnichannel Commerce Core?

The Challenge

nopCommerce today
operates as an isolated online storefront.

In real enterprise contexts,
order management, stock, fulfillment and
shipping live in specialist systems.

The goal:
Evolve nopCommerce into a
Commerce Coordination Core
that integrates ERP, WMS, and Shipping
with resilience and consistency.

Strategic Goals

1 **Unified Commerce Core**

nopCommerce acts as coordinator, not just a storefront

2 **Cross-channel Visibility**

Stock, order state and fulfillment visible across all channels

3 **Resilience to External Failures**

System stays useful when WMS, shipping or ERP degrade

02 · CURRENT STATE

nopCommerce Today

Layered Architecture

Presentation Layer

Web UI + HTTP Controllers

Application / Services Layer

Business logic: Orders, Catalog, Customers

Domain Layer

Core entities and business models

Data Access Layer

DB persistence (single SQL Server)

✓ Strengths

- Full order cycle control — internally coherent
- Mature, stable platform with rich feature set
- Modular internal structure (extensible plugins)

✗ Limitations vs Scenario

- Synchronous-only — no async event flows
- No native integration with ERP / WMS / Shipping
- Tightly coupled — hard to extract integration logic
- Assumes sole ownership of stock & order state

Bounded Contexts & Ownership

nopCommerce

Commerce Experience Context

Catalog, Cart, Checkout, Order status UI

Order Management Context

Order creation, state, event emission

events

Order Integration Context

NEW

Async coordination, retries, eventual consistency

Warehouse Context (WMS)

NEW

Real stock, product reservation, fulfillment prep

Shipping Context

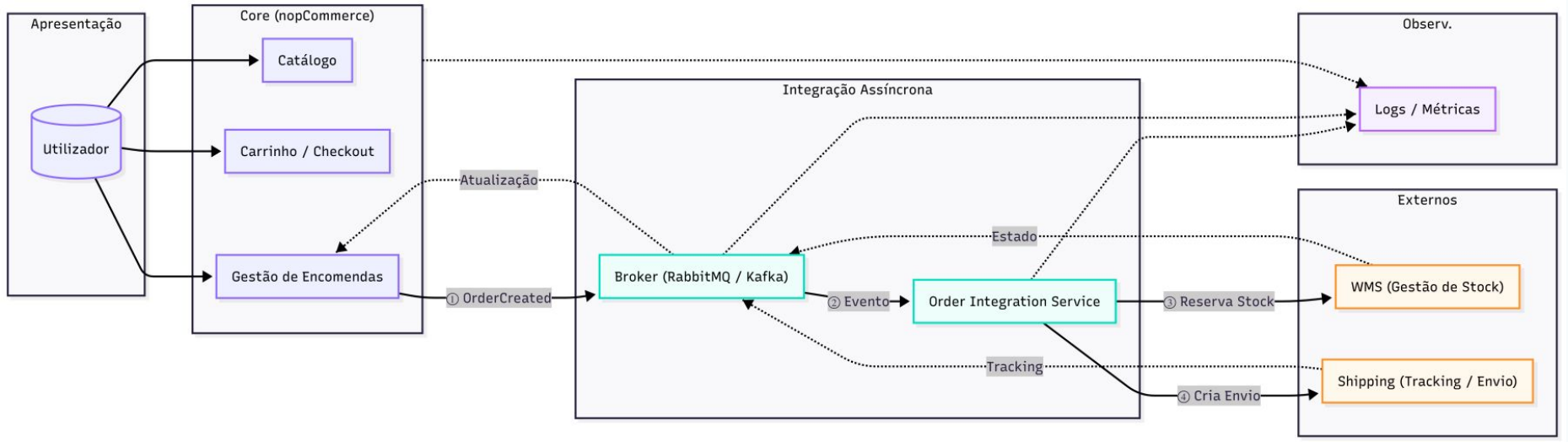
NEW

Shipment processing, tracking, delivery state

5 Quality Attribute Scenarios

ID	Attribute	Stimulus	System Response	Metric
Q A1	**Resilience**	WMS unavailability	Order saved → intermediate state → async retry	Zero order loss; auto-recovery
Q A2	**Eventual Consistency**	Shipping delay	Intermediate state shown; updated when info arrives	Final consistent state in minutes
Q A3	**Decoupling**	New external system	Integrated via dedicated component, zero core changes	Localized changes; reduced integration time
Q A4	**Observability**	External comm failure	Distributed tracing + metrics + logs across all flows	Root cause identified quickly
Q A5	**Continuity**	Delay during checkout	Order completes; external ops processed asynchronously	No user-facing interruption

Architecture Overview



Interactions & Data Ownership

Synchronous Flows

Remain inside nopCommerce
(user-facing, requires immediate response)

- Catalog browsing & search
- Shopping cart operations
- Checkout process
- Order submission
- Order status query by customer

Asynchronous Flows

Via Message Broker
(decoupled, resilient, eventually consistent)

- Order created event → Broker
- WMS: stock reservation & fulfillment
- Shipping: shipment creation & tracking
- ERP: back-office order state sync
- Delivery status update → nopCommerce

Architectural Decision Records

ADR 1 — Adopt Async Communication

Context: WMS/Shipping integrations are subject to delays and failures — synchronous calls in critical flows create unacceptable coupling.

Decision: Introduce a Message Broker for all nopCommerce ↔ external system communication.

✗ Rejected: Direct sync calls | Internal polling

✓ Because: Reduces coupling · tolerates degradation · supports eventual consistency

ADR 2 — Dedicated Order Integration Service

Context: Omnichannel coordination requires retry logic, correlation, failure handling — unfit to live inside nopCommerce core.

Decision: Extract an independent Order Integration Service as the sole orchestrator of external flows.

✗ Rejected: Logic inside nopCommerce | Per-system direct integration

✓ Because: Single responsibility · future systems added without core changes · isolated evolution

ADR 3 — Separate Data Ownership per Context

Context: Multiple contexts (Commerce, Integration, WMS, Shipping) risk tight coupling and coordination problems if sharing a DB.

Decision: Each bounded context owns its own data store — no shared database across extracted service boundaries.

✗ Rejected: Shared SQL database | Cross-context direct queries

✓ Because: Enforces boundaries · independent evolution · aligned with assignment constraints

Design Framework: ADD

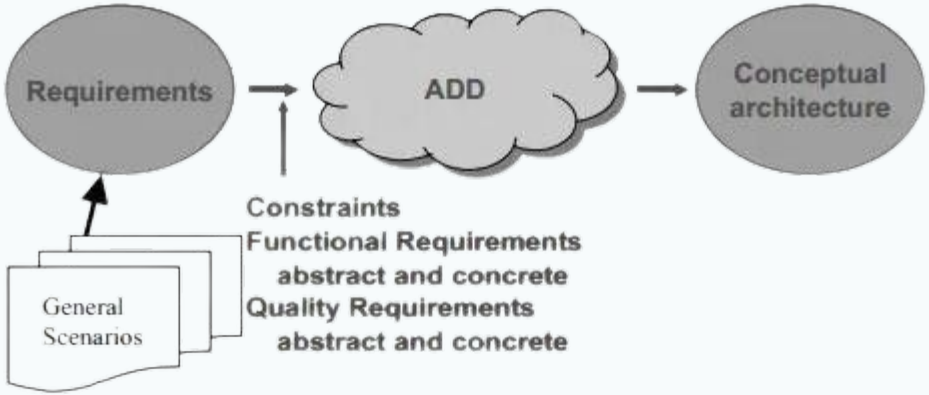
Attribute-Driven Design (ADD)

ADD starts from Quality Attribute Scenarios and uses them to drive every structural decision.

This fits our project because:

- We start from concrete QAs (resilience, observability, decoupling)
- We are evolving, not rewriting — ADD guides focused change
- Decisions map directly from QA → design element
- Clear traceability: drivers → QAs → architecture → ADRs

Why not ACDM or ADM?
- ACDM: better for change analysis over existing, complex systems
- ADM (TOGAF): better suited for full enterprise architecture transformations



07 · RISK PLAN

Risks & Mitigation Strategies

ID	Risk	Level	Description	Mitigation
R1	**Async Complexity**	HIGH	Event ordering, failure handling increases system complexity	Implement simple event flow first; validate correct message processing
R2	**Temporary Inconsistency**	MED	Eventual consistency may surface stale state to the user	Simulate external delays; verify intermediate states are correctly shown
R3	**External Reliability**	MED	WMS / Shipping failures impact order processing	Simulate failures; verify retry + idempotency; no message loss
R4	**Observability Gaps**	LOW	Distributed flows are hard to trace without proper instrumentation	Add distributed tracing, logs & metrics; trace full order flow

5-Phase Evolution Roadmap

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5
Observability	**Message Broker**	**Order Integration Service**	**External Systems**	**Eventual Consistency**
Logs, metrics, distributed tracing inside nopCommerce	Introduce RabbitMQ/Kafka; async event infrastructure	Build dedicated integration component; simple order flow	Connect WMS & Shipping; simple then complex flows	Intermediate order states; async sync with all external systems

08 · FEASIBILITY SPIKE

Validating the Riskiest Assumption

What We're Testing

Riskiest assumption:
Async communication between nopCommerce and external components.

Spike Flow:
1. nopCommerce emits order created event
2. Event published to Message Broker
3. Simulated Order Integration Service consumes it
4. OIS simulates external response (success / failure)
5. Retry and error handling verified
6. Logs, metrics and tracing observed

Expected Results

✓ **Event correctness**

Events correctly emitted and consumed by OIS

✓ **Decoupling verified**

Core system unaffected by external component state

✓ **Failure behavior**

System behaves correctly under simulated failures

✓ **Retry & idempotency**

Retry mechanisms work; no duplicate processing

✓ **Observability**

Full flow visible through logs, metrics & traces

Thank you

Questions & Discussion